FedAPM: Federated Learning via ADMM with Partial Model Personalization

Shengkun Zhu
School of Computer Science
Wuhan University
Wuhan, China
whuzsk66@whu.edu.cn

Feiteng Nie
School of Computer Science
Wuhan University
Wuhan, China
niefeiteng@whu.edu.cn

Jinshan Zeng*
School of Management
Xi'an Jiaotong University
Xi'an, China
jsh.zeng@gmail.com

Sheng Wang*
School of Computer Science
Wuhan University
Wuhan, China
swangcs@whu.edu.cn

Yuan Sun
La Trobe Business School
La Trobe University
Melbourne, Australia
yuan.sun@latrobe.edu.au

Yuan Yao
Hong Kong University of Science and Technology
Hong Kong, China
yuany@ust.hk

Shangfeng Chen
School of Computer Science
Wuhan University
Wuhan, China
brucechen@whu.edu.cn

Quanqing Xu
Oceanbase
Ant group
Hangzhou, China
xuquanqing.xqq@oceanbase.com

Chuanhui Yang
Oceanbase
Ant group
Hangzhou, China
rizhao.ych@oceanbase.com

Abstract

In federated learning (FL), the assumption that datasets from different devices are independent and identically distributed (i.i.d.) often does not hold due to user differences, and the presence of various data modalities across clients makes using a single model impractical. Personalizing certain parts of the model can effectively address these issues by allowing those parts to differ across clients, while the remaining parts serve as a shared model. However, we found that *partial model personalization may exacerbate client drift* (each client's local model diverges from the shared model), thereby reducing the effectiveness and efficiency of FL algorithms. We propose an FL framework based on the alternating direction method of multipliers (ADMM), referred to as `FedAPM`, to mitigate client drift. We construct the augmented Lagrangian function by incorporating first-order and second-order proximal terms into the objective, with the second-order term providing fixed correction and the first-order term offering compensatory correction between the local and shared models. Our analysis demonstrates that `FedAPM`, by using explicit estimates of the Lagrange multiplier, is more stable and efficient in terms of convergence compared to other FL frameworks. We establish the global convergence of `FedAPM` training from arbitrary initial points to a stationary point, achieving three types of rates: constant, linear, and sublinear, under mild assumptions. We conduct experiments using four heterogeneous and multimodal datasets with different metrics to validate the performance of `FedAPM`. Specifically, `FedAPM` achieves faster and more accurate convergence, outperforming the SOTA methods with average improvements of 12.3% in test accuracy, 16.4% in F1 score, and 18.0% in AUC while requiring fewer communication rounds.

CCS Concepts

• **Computing methodologies → Distributed artificial intelligence**; *Distributed algorithms*; Regularization.

Keywords

Federated learning, partial model personalization, ADMM, client drift, global convergence.

ACM Reference Format:
Shengkun Zhu, Feiteng Nie, Jinshan Zeng*, Sheng Wang*, Yuan Sun, Yuan Yao, Shangfeng Chen, Quanqing Xu, and Chuanhui Yang. 2025. FedAPM: Federated Learning via ADMM with Partial Model Personalization. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 28 pages. https://doi.org/10.1145/3711896.3736954

*Corresponding authors: Sheng Wang and Jinshan Zeng.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, August 3–7, 2025, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3736954

1 Introduction

With the widespread use of mobile devices, vast amounts of data are generated that fuel various machine learning applications [65, 68, 70]. However, traditional cloud-based training methods face issues of privacy leakage and data transmission costs [29, 62]. Additionally, with the implementation of privacy regulations like the California Consumer Privacy Act (CCPA) [1] and General Data Protection Regulation (GDPR) [2], protecting user data has become even more critical. Federated learning (FL) is a distributed machine learning framework that protects privacy by enabling local model

training without the need for data centralization [40, 48], addressing challenges like data silos and insufficient samples [30, 38], and fostering advancements in various intelligent applications [35, 72].

In FL, it is typically assumed that the datasets from various devices are sampled from the same or highly similar distributions [38, 41, 46]. However, due to the distinct characteristics of users and the growing prominence of personalized on-device services, the assumption of independent and identically distributed (i.i.d.) data often does not hold in practical scenarios [55, 71, 78]. This leads to statistical heterogeneity, which can significantly impact the effectiveness of FL algorithms [71]. Moreover, different clients may have data in various modalities [15, 16, 24], such as images, videos, text, and audio, each requiring distinct models like convolutional neural networks (CNNs) for images [37] and recurrent neural networks (RNNs) for audio [26]. Therefore, relying on a single model for all cases is ineffective and impractical [52, 63].

Partial model personalization [52] is a method designed to address the challenge of heterogeneous and multimodal data by dividing the model parameters into two groups: shared and personalized model parameters. The dimensions of personalized models can vary among clients, enabling personalized components to differ in the number of parameters or even in their architecture, while the shared model is common among clients and maintains the same structure. However, while personalized models can effectively address the impact of heterogeneous and multimodal data, the shared model cannot, leading to the common saying: *it started with a bang and ended with a whimper*. This limitation arises because each client's local objectives differ. When solving for the shared model, the result is often a stationary point with respect to the local objective, rather than a stationary point with respect to the global objective. This issue is commonly referred to as *client drift* [32].

Several studies consider using full model personalization to address client drift [23, 39, 43], which involves personalizing all model parameters. However, each model requires twice the memory footprint of the full model, which limits the size of trainable models. Moreover, Pillutla et al. [52] proposed that full model personalization may not be essential for modern deep learning architectures, which consist of multiple simple functional units often structured in layers or other complex interconnected designs. Focusing on personalizing the appropriate components, guided by domain expertise, can yield significant advantages with only a minimal increase in memory requirements. Current partial model personalization methods can be categorized into two types based on their iterative approach: one using Gauss-Seidel iteration [19, 52, 60], and the other employing Jacobi iteration [4, 27, 42], referred to as `FedAlt` and `FedSim`, respectively. Pillutla et al. [52] theoretically demonstrated the convergence of both methods and experimentally validated that `FedAlt` achieves better test performance than `FedSim`. However, these methods do not fundamentally address client drifts, as multiple local updates can still lead to deviations of each client's local model from the optimal representation. Moreover, we find that partial model personalization sometimes even exacerbates client drift, as shown in Figure 1. The local updates in `FedAlt` and `FedSim`, compared to `FedAvg` (a de facto FL framework without personalization) [48], bring the local model closer to the optimal local model while moving it further from the optimal shared model.

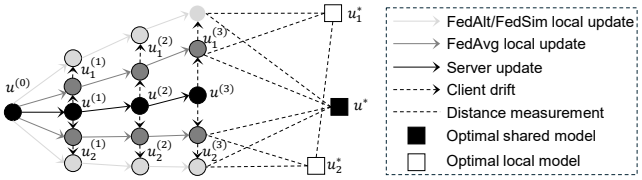


Figure 1: Client drift in `FedAvg` [48] and `FedAlt`/`FedSim` [52] is illustrated using two clients with three local update steps. Partial model personalization reduces the gap between u_i and u_i^*, but increases the gap between u_i and u^*. This may cause the shared model u to drift further from u^*, as $u = (u_1 + u_2)/2$.

Our main contribution in this paper is to address the issue that partial model personalization can exacerbate client drift. Specifically, we propose an FL framework based on the alternating direction method of multipliers (ADMM): `FedAPM`. We construct the augmented Lagrangian function by incorporating first-order and second-order proximal terms into the objective. The second-order term offers a fixed correction, while the first-order term provides a compensatory adjustment to fine-tune the local update. Moreover, we demonstrate that `FedAPM` is more stable and efficient in terms of convergence compared to `FedAlt` and `FedSim`, which results in improved communication efficiency. Specifically, we show that `FedAlt` and `FedSim` can be interpreted as using the inexact penalty method, while `FedAPM` adopts ADMM, an augmented Lagrangian method (also known as the exact penalty method). In `FedAPM`, explicit estimates of the Lagrange multiplier are used to address the ill-conditioning that is inherent in quadratic penalty functions [51].

We theoretically establish the global convergence of `FedAPM` under weaker assumptions than those used in [19, 27, 52]. Specifically, we establish global convergence with constant, linear, and sublinear convergence rates based on the *Kurdyka-Łojasiewicz (KŁ)* [36] *inequality framework*, as formulated in [6, 67]. Our convergence analysis differs from existing work [6, 67] in several key ways, enabling us to achieve the previously mentioned convergence results. According to [6, 67], the conditions of *sufficient descent*, *relative error*, *continuity*, and the *KŁ property* guarantee the global convergence of a nonconvex algorithm. In contrast, we demonstrate sufficient descent and relative error conditions within an inexact ADMM optimization framework, while existing theories [6, 67] require that all subproblems in ADMM be solved exactly. Our theoretical contributions are also of value to the optimization community.

Our experimental contributions demonstrate that partial model personalization can exacerbate client drift, while `FedAPM` effectively alleviates this issue. Specifically, we validate `FedAPM` on several real-world datasets (including heterogeneous and multimodal data) using three partial model personalization strategies, four baselines, and six metrics. We demonstrate that `FedAPM` improves the overall performance of the SOTA methods, resulting in an average improvement of 12.3%, 16.4%, and 18.0% in testing accuracy, F1 score, and AUC, respectively. In terms of communication efficiency, `FedAPM` requires fewer communication rounds to converge to a lower loss. Moreover, we explore the impact of hyperparameters on `FedAPM` and provide adjustment strategies based on empirical findings.

2 Related Work

Since partial model personalization is a special case of personalized FL approaches, we first review the existing personalized FL methods. Subsequently, given that ADMM is a primal-dual-based optimization framework, we review the current research that utilizes primal-dual-based methods as solvers for FL.

Personalized FL. According to the model partitioning strategies, PFL can be categorized into two types: full model personalization and partial model personalization [52]. Since the former is not the focus of this paper, interested readers can refer to [23, 39, 43, 81]. Here, we will only review partial model personalization methods. Based on the iterative methods (Gauss-Seidel and Jacobi), partial model personalization can be divided into two categories: `FedAlt` [19, 60] and `FedSim` [4, 27, 42]. These methods proposed different schemes for personalizing model layers. Liang et al. [42] and Collins et al. [19] proposed to personalize the input layers to learn a personalized representation, while Arivazhagan et al. [4] proposed learning a shared representation across various tasks through output layer personalization. However, regardless of which part of the model is personalized, the shared part still suffers from client drift, and this phenomenon can sometimes be exacerbated, leading to a decline in overall model performance. In terms of theory, Pillutla et al. [52] provided convergence guarantees for these methods, demonstrating a sublinear convergence rate under the assumptions of *smoothness, bounded variance, and partial gradient diversity*, which are often too strong in practical scenarios.

Primal-dual-based FL. Existing FL frameworks can be classified into three categories based on the type of variables being solved: primal-based FL [32, 41, 48], dual-based FL [47, 61, 62], and primal-dual-based FL [25, 31, 66, 77, 79–81]. Primal-based FL solves the primal problem of FL, which makes it easy to implement [81]. Consequently, it has become one of the most widely used FL frameworks [10, 12, 44]. In contrast, dual-based FL solves dual problems and has been shown to have better convergence than primal-based FL [56, 57]. However, dual-based FL is only applicable to convex problems. In recent years, primal-dual-based FL has gained more attention due to its advantages from both primal-based and dual-based methods. ADMM, as an exact penalty method [11, 74], has been applied in FL optimization in recent years [79]. It has been shown to offer advantages in terms of convergence [80] and alleviating data heterogeneity [25]. However, these studies do not explain why applying ADMM in FL outperforms other FL frameworks, a question we will discuss from an optimization perspective.

3 Preliminaries

This section presents the notations used throughout the paper and provides detailed definitions of both FL and ADMM. Moreover, we present three types of partial model personalization methods.

3.1 Notations

We use different text-formatting styles to represent different mathematical concepts: plain letters for scalars, bold letters for vectors, and capitalized letters for matrices. For instance, m represents a scalar, $\boldsymbol{v}$ represents a vector, and $\boldsymbol{V}$ denotes a matrix. Without loss of generality, all training models in this paper are represented using vectors. We use $[m]$ to represent the set $\{1, 2, ..., m\}$. We use ":=" to indicate a definition, while $\mathbb{R}^d$ represents the d-dimensional Euclidean space. We represent the inner product of vectors, such as $\langle \boldsymbol{u}, \boldsymbol{v} \rangle$, as the sum of the products of their corresponding elements. We use $||\cdot||$ to denote the Euclidean norm of a vector. Table 1 enumerates the notations used in this paper along with the description.

Table 1: Summary of notations

Notations	Description
$\boldsymbol{X}_i, i \in [m]$	The local dataset
$\boldsymbol{v}_i, i \in [m]$	The personalized model
$\boldsymbol{u}_i, i \in [m]$	The local model
$\boldsymbol{\pi}_i, i \in [m]$	The dual variable
$\alpha_i, i \in [m]$	The weight parameter
$\boldsymbol{u}$	The shared model
ρ	The penalty parameter
$\mathcal{S}^t$	The selected clients set in the t-th iteration

3.2 Federated Learning

In an FL scenario involving m clients, each client i holds a local dataset $\boldsymbol{X}_i$ consisting of n_i data samples drawn from the distribution $\mathcal{D}_i$. These clients collaborate through a central server to jointly train a model $\boldsymbol{u}$ that minimizes empirical risk as follows [48]:

$$\min_{\boldsymbol{u}} \left\{ \sum_{i=1}^{m} \alpha_i f_i(\boldsymbol{u}) := \sum_{i=1}^{m} \alpha_i \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_i}[\ell_i(\boldsymbol{u};\boldsymbol{x})] \right\}, \tag{1}$$

where α_i is a weight parameter, typically chosen as either $1/m$ or n_i/n, where $n = \sum_{i=1}^{m} n_i$ represents the total data number, $\boldsymbol{x}$ denotes a random sample from $\mathcal{D}_i$, $\ell_i(\boldsymbol{u};\boldsymbol{x})$ is the loss function for the model $\boldsymbol{u}$ with respect to the sample $\boldsymbol{x}$, and $f_i(\boldsymbol{u}) := \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_i}[\ell_i(\boldsymbol{u};\boldsymbol{x})]$ represents the expected loss over the data distribution $\mathcal{D}_i$. In this paper, we consider $f_i(\boldsymbol{u})$ to be possibly non-convex.

3.3 Alternating Direction Method of Multipliers

ADMM is an optimization method within the augmented Lagrangian framework, ideally suited for addressing the following problem [11]:

$$\min_{\boldsymbol{u}\in\mathbb{R}^r, \boldsymbol{v}\in\mathbb{R}^q} f(\boldsymbol{u}) + g(\boldsymbol{v}), \quad \text{s.t. } A\boldsymbol{u} + B\boldsymbol{v} - \boldsymbol{b} = \boldsymbol{0},$$

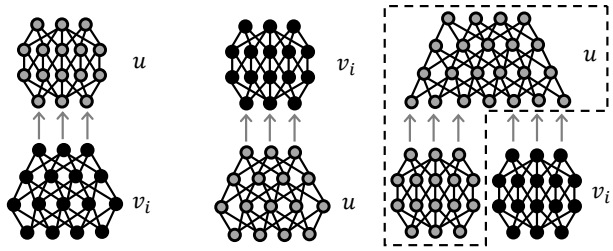
where $A \in \mathbb{R}^{p\times r}$, $B \in \mathbb{R}^{p\times q}$, and $\boldsymbol{b} \in \mathbb{R}^p$. We directly give the augmented Lagrangian function of the problem as follows,

$$\mathcal{L}(\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{\pi}) := f(\boldsymbol{u}) + g(\boldsymbol{v}) + \langle \boldsymbol{\pi}, A\boldsymbol{u} + B\boldsymbol{v} - \boldsymbol{b} \rangle + \frac{\rho}{2}||A\boldsymbol{u} + B\boldsymbol{v} - \boldsymbol{b}||^2,$$

where $\boldsymbol{\pi} \in \mathbb{R}^p$ is the dual variable, and $\rho > 0$ is the penalty parameter. After initializing the variables with $(\boldsymbol{u}^0, \boldsymbol{v}^0, \boldsymbol{\pi}^0)$, ADMM iteratively performs the following steps:

$$\begin{cases} \boldsymbol{u}^{t+1} = \operatorname{argmin}_{\boldsymbol{u}\in\mathbb{R}^r} \mathcal{L}(\boldsymbol{u}, \boldsymbol{v}^t, \boldsymbol{\pi}^t), \\ \boldsymbol{v}^{t+1} = \operatorname{argmin}_{\boldsymbol{v}\in\mathbb{R}^q} \mathcal{L}(\boldsymbol{u}^{t+1}, \boldsymbol{v}, \boldsymbol{\pi}^t), \\ \boldsymbol{\pi}^{t+1} = \boldsymbol{\pi}^t + \rho(A\boldsymbol{u}^{t+1} + B\boldsymbol{v}^{t+1} - \boldsymbol{b}). \end{cases}$$

ADMM offers effective distributed and parallel computing capabilities, efficiently addresses equality-constrained problems, and guarantees global convergence [67]. This makes it especially suitable



(a) Input personalization (b) Output personalization (c) Split input personalization

Figure 2: Three examples of partial model personalization in deep neural networks, where v_i and u represent the personalized and shared models, respectively.

for large-scale optimization tasks and widely used in distributed computing [11] and machine learning [74].

3.4 Partial Model Personalization

Partial model personalization allows clients to make personalized adjustments based on the shared global model. Pillutla et al. [52] categorized current methods for partial model personalization in deep neural networks into three types based on their applications:

- **Input personalization**: The lower layers are trained locally, and the upper layers are shared among clients.
- **Output personalization**: The upper layers are trained locally, and the lower layers are shared among clients.
- **Split input personalization**: The input layers are divided horizontally into a shared and a personal part, which processes different portions of the input vector, and their outputs are concatenated before being passed to the upper layers of the model.

In Figure 2, we present examples of three partial model personalization strategies. Input personalization can be seen as learning a shared representation among different clients [19, 42]. Despite heterogeneous data with different labels, tasks may share common features, like those in various image types or word-prediction tasks [7, 37]. After learning this representation, each client's labels can be predicted with a linear classifier or shallow neural network [19]. Output personalization is applicable to various tasks [4], such as personalized image aesthetics [53] and personalized highlight detection [20], where explicit user features are absent from the data and must be inferred during training. As a result, the same input data may receive different labels from different users, indicating that personalized models must vary across users to accurately predict distinct labels for similar test data. Split input personalization helps protect private user features by localizing their personalized embeddings on the device [52]. Similar architectures have been proposed for context-dependent language models [49].

4 Proposed FedAPM

In this section, we begin by presenting the formulation of the optimization problem and defining the stationary points. Next, we offer a detailed algorithmic description of FedAPM. Finally, we discuss the advantages of FedAPM compared to other FL frameworks.

4.1 Problem Formulation

Let $\boldsymbol{V} := \{\boldsymbol{v}_i\}_{i=1}^m$ be a set of personalized models and $\boldsymbol{u}$ be the shared model. We consider solving the following optimization problem:

$$\min_{\boldsymbol{V},\boldsymbol{u}} \Big\{ f(\boldsymbol{V},\boldsymbol{u}) := \sum_{i=1}^m \alpha_i f_i(\boldsymbol{v}_i,\boldsymbol{u}) \Big\}, \tag{2}$$

where $f_i(\boldsymbol{v}_i,\boldsymbol{u}) := \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}_i}[\ell_i((\boldsymbol{v}_i,\boldsymbol{u});\boldsymbol{x})]$ denotes the expected loss over the data distribution $\mathcal{D}_i$ for the combined model $(\boldsymbol{v}_i,\boldsymbol{u})$. The existing two types of methods for solving (2) are FedAlt and FedSim [52], which are analogous to the Gauss-Seidel and Jacobi updates in numerical linear algebra [21], respectively. However, we found that these two methods can exacerbate client drift in the shared model (see Figure 1, which will be validated in Section 6). To address this issue, we use ADMM to solve (2), and we discuss the advantages of using ADMM in Section 4.4. We introduce auxiliary variables $\boldsymbol{U} := \{\boldsymbol{u}_i\}_{i=1}^m$ to transform (2) into a separable form (with respect to a partition or splitting of the variable into multi-block variables):

$$\min_{\boldsymbol{V},\boldsymbol{U},\boldsymbol{u}} \Big\{ f(\boldsymbol{V},\boldsymbol{U}) := \sum_{i=1}^m \alpha_i f_i(\boldsymbol{v}_i,\boldsymbol{u}_i) \Big\}, \quad \text{s.t. } \boldsymbol{u}_i = \boldsymbol{u},\ i\in[m]. \tag{3}$$

Note that (3) is equivalent to (2) in the sense that the optimal solutions coincide (discussed in Section 4.2). To apply ADMM for (3), we define the augmented Lagrangian function as follows:

$$\begin{aligned} \mathcal{L}(\boldsymbol{V},\boldsymbol{U},\boldsymbol{\Pi},\boldsymbol{u}) &:= \sum_{i=1}^m \mathcal{L}_i(\boldsymbol{v}_i,\boldsymbol{u}_i,\boldsymbol{\pi}_i,\boldsymbol{u}), \\ \mathcal{L}_i(\boldsymbol{v}_i,\boldsymbol{u}_i,\boldsymbol{\pi}_i,\boldsymbol{u}) &:= \alpha_i f_i(\boldsymbol{v}_i,\boldsymbol{u}_i) + \langle \boldsymbol{\pi}_i, \boldsymbol{u}_i - \boldsymbol{u}\rangle + \frac{\rho}{2}\|\boldsymbol{u}_i-\boldsymbol{u}\|^2, \end{aligned} \tag{4}$$

where $\Pi := \{\boldsymbol{\pi}_i\}_{i=1}^m$ denotes the set of dual variables, and $\rho > 0$ represents the penalty parameter. The ADMM algorithm to solve (3) can be outlined as follows: starting from arbitrary initialization $(\boldsymbol{V}^0,\boldsymbol{U}^0,\Pi^0,\boldsymbol{u}^0)$, the update is iteratively performed for each $t \ge 0$:

$$\begin{cases} \boldsymbol{v}_i^{t+1} = \operatorname{argmin}_{\boldsymbol{v}_i} \mathcal{L}_i(\boldsymbol{v}_i,\boldsymbol{u}_i^t,\boldsymbol{\pi}_i^t,\boldsymbol{u}^t), \\ \boldsymbol{u}_i^{t+1} = \operatorname{argmin}_{\boldsymbol{u}_i} \mathcal{L}_i(\boldsymbol{v}_i^{t+1},\boldsymbol{u}_i,\boldsymbol{\pi}_i^t,\boldsymbol{u}^t), \\ \boldsymbol{\pi}_i^{t+1} = \boldsymbol{\pi}_i^t + \rho(\boldsymbol{u}_i^{t+1} - \boldsymbol{u}^t), \\ \boldsymbol{u}^{t+1} = \operatorname{argmin}_{\boldsymbol{u}} \mathcal{L}(\boldsymbol{V}^{t+1},\boldsymbol{U}^{t+1},\Pi^{t+1},\boldsymbol{u}) \\ \qquad\quad = \dfrac{1}{m}\displaystyle\sum_{i=1}^m (\boldsymbol{u}_i^{t+1} + \frac{1}{\rho}\boldsymbol{\pi}_i^{t+1}). \end{cases} \tag{5}$$

Due to the potentially non-convex nature of f_i, closed-form solutions for $\boldsymbol{u}_i$ and $\boldsymbol{v}_i$ may not exist. We will provide an approach to address this issue later in Section 4.3.

4.2 Stationary Points

We define the optimal conditions of (3) as follows:

Definition 1 (Stationary point)**.** *A point* $(\boldsymbol{V}^*,\boldsymbol{U}^*,\boldsymbol{u}^*,\Pi^*)$ *is a stationary point of* (3) *if it satisfies*

$$\begin{cases} \alpha_i \nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_i^*,\boldsymbol{u}_i^*) + \boldsymbol{\pi}_i^* + \rho(\boldsymbol{u}_i^* - \boldsymbol{u}^*) = 0, & i\in[m], \\ \nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i^*,\boldsymbol{u}_i^*) = 0, & i\in[m], \\ \boldsymbol{u}_i^* - \boldsymbol{u}^* = 0, & i\in[m], \\ \displaystyle\sum_{i=1}^m \boldsymbol{\pi}_i^* = 0. \end{cases} \tag{6}$$

Algorithm 1: FedAPM

Input: T : communication rounds, ρ: penalty parameter, m: number of clients, $\boldsymbol{X}_i$: local dataset, σ_i: hyperparameter.
Output: $\{\boldsymbol{v}_i\}_{i=1}^m$ (personalized), $\{\boldsymbol{u}_i\}_{i=1}^m$ (local), $\boldsymbol{u}$ (shared).

1 **Initialize:** $\boldsymbol{v}_i^0, \boldsymbol{u}_i^0, \boldsymbol{\pi}_i^0, \boldsymbol{z}_i^0 = \boldsymbol{u}_i^0 + \frac{1}{\rho}\boldsymbol{\pi}_i^0, \xi_i^0, \mu_i \in (0,1), i \in [m]$.
2 **for** $t = 0, 1, \cdots, T-1$ **do**
3 *Weights upload:* All clients send $\boldsymbol{z}_i^t$ to the server ;
4 *Weights average:* Server aggregates $\boldsymbol{z}_i^t$ by

$$\boldsymbol{u}^t = \frac{1}{m}\sum_{i=1}^m \boldsymbol{z}_i^t. \tag{8}$$

5 *Weights feedback:* Broadcast $\boldsymbol{u}^t$ to all clients;
6 *Client selection:* Randomly select s clients $\mathcal{S}^t \subset [m]$;
7 **for** *each client* $i \in \mathcal{S}^t$ **do**
8 *Local update:* client i update its parameters as follows:

$$\boldsymbol{v}_i^{t+1} = \operatorname{argmin}_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_i^t) + \frac{\sigma_i}{2}\|\boldsymbol{v}_i - \boldsymbol{v}_i^t\|^2, \tag{9}$$

$$\xi_i^{t+1} \le \mu_i \xi_i^t, \tag{10}$$

$$\text{Find a } \xi_i^{t+1}\text{-approximate solution } \boldsymbol{u}_i^{t+1} \text{ (Definition 2)}, \tag{11}$$

$$\boldsymbol{\pi}_i^{t+1} = \boldsymbol{\pi}_i^t + \rho(\boldsymbol{u}_i^{t+1} - \boldsymbol{u}^t), \tag{12}$$

$$\boldsymbol{z}_i^{t+1} = \boldsymbol{u}_i^{t+1} + \frac{1}{\rho}\boldsymbol{\pi}_i^{t+1} \tag{13}$$

9 **for** *each client* $i \notin \mathcal{S}^t$ **do**
10 *Local invariance:* client i keep its parameters as follows:

$$(\zeta_i^{t+1}, \boldsymbol{v}_i^{t+1}, \boldsymbol{u}_i^{t+1}, \boldsymbol{\pi}_i^{t+1}, \boldsymbol{z}_i^{t+1}) = (\zeta_i^t, \boldsymbol{v}_i^t, \boldsymbol{u}_i^t, \boldsymbol{\pi}_i^t, \boldsymbol{z}_i^t). \tag{14}$$

11 **return** $\{\boldsymbol{v}_i^T\}_{i=1}^m$, $\{\boldsymbol{u}_i^T\}_{i=1}^m$, $\boldsymbol{u}^T$.

If f_i is convex with respect to $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$ for any $i \in [m]$, then a point is a globally optimal solution if and only if it satisfies (6). Moreover, a stationary point $(\boldsymbol{V}^*, \boldsymbol{U}^*, \boldsymbol{u}^*, \Pi^*)$ of (3) indicates that

$$\begin{cases} \sum_{i=1}^m \alpha_i \nabla_{\boldsymbol{u}} f_i(\boldsymbol{v}_i^*, \boldsymbol{u}^*) = 0, \\ \nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i^*, \boldsymbol{u}^*) = 0, \quad i \in [m], \end{cases} \tag{7}$$

That is, $(\boldsymbol{V}^*, \boldsymbol{u}^*)$ is also a stationary point of (2).

4.3 Algorithmic Design

In Algorithm 1, we present the details of FedAPM. Figure 3 shows a running example of FedAPM. The algorithm is divided into two parts, which are executed on the clients and the server, respectively.

- **Server update:** First, the clients upload their locally computed parameters $\boldsymbol{z}_i^t$ to the server (Line 3). The server aggregates these parameters to obtain the shared model $\boldsymbol{u}^t$ (Line 4), and then broadcasts $\boldsymbol{u}^t$ to each client (Line 5). Next, the clients are randomly divided into two groups (Line 6). For the clients in $\mathcal{S}^t$, they perform a local update, while the clients not in $\mathcal{S}^t$ keep their local parameters unchanged.
- **Client update:** The clients in $\mathcal{S}^t$ update the model parameters locally (Line 8). Due to the possible non-convex nature of f_i, we propose using proximal update strategies for solving $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$. Specifically, we consider solving $\boldsymbol{v}_i$ by adding a proximal term to the augmented Lagrangian function such that the new function is convex with respect to $\boldsymbol{v}_i$, that is

$$\boldsymbol{v}_i^{t+1} = \operatorname{argmin}_{\boldsymbol{v}_i} \mathcal{L}_i(\boldsymbol{v}_i, \boldsymbol{u}_i^t, \boldsymbol{\pi}_i^t, \boldsymbol{u}^t) + \frac{\sigma_i}{2}\|\boldsymbol{v}_i - \boldsymbol{v}_i^t\|^2, \tag{15}$$

where σ_i is a hyperparameter employed to control the degree of approximation between $\boldsymbol{v}_i$ and $\boldsymbol{v}_i^t$. The purpose of using proximal update strategies is to effectively stabilize the training process [73]. For each subproblem involving $\boldsymbol{v}_i$, we assume the minimizer can be achieved. In a similar vein, to solve for $\boldsymbol{u}_i$, we consider applying *stochastic gradient descent* (SGD) to obtain an approximate solution for $\boldsymbol{u}_i$, defined as follows:

Definition 2 (ξ-Approximate solution)**.** *For $\xi_i^{t+1} \in (0,1)$, we say $\boldsymbol{u}_i^{t+1}$ is a ξ_i^{t+1}-approximate solution of $\min_{\boldsymbol{u}_i} \mathcal{L}_i(\boldsymbol{v}_i^{t+1}, \boldsymbol{u}_i, \boldsymbol{\pi}_i^t, \boldsymbol{u}^t)$ if*

$$\|\alpha_i \nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_i^{t+1}, \boldsymbol{u}_i^{t+1}) + \pi_i^t + \rho(\boldsymbol{u}_i^{t+1} - \boldsymbol{u}^t)\|^2 \le \xi_i^{t+1}. \tag{16}$$

Note that a smaller ξ_i^{t+1} corresponds to higher accuracy.

SGD is commonly used to compute approximate solutions in a finite number of iterations and is applied in many FL frameworks [23, 41, 79]. Note that we set the local accuracy level at each iteration as $\xi_i^{t+1} \le \mu_i \xi_i^t$. In the early stages of algorithm iteration, when the solution is far from stationary points, setting a larger ξ_i can effectively reduce the number of iterations and improve efficiency. As iterations increase, the solution gets closer to the stationary points. Therefore, reducing ξ_i can improve the accuracy. After computing $\boldsymbol{v}_i$ and $\boldsymbol{u}_i$, we follow from (5) to update the dual variable $\boldsymbol{\pi}_i$ and calculate the update messages $\boldsymbol{z}_i$ that will be uploaded to the server. For those clients that are not in $\mathcal{S}^t$, the local parameters remain unchanged (Line 10).

4.4 Discussions

In this section, we provide an intuitive discussion on why partial model personalization can exacerbate client drift in the shared model. Furthermore, we highlight the superiority of FedAPM in addressing client drift, as well as its improved convergence stability and efficiency compared to FedAlt and FedSim.

In input personalization, the personalized module processes the raw input before it reaches the shared model. As a result, each client's personalized transformation alters the input distribution seen by the shared model. Due to heterogeneity in client data and personalized modules, the shared model receives inconsistent and client-specific inputs, which leads to misaligned optimization objectives across clients. This discrepancy causes the shared model to drift, as it is effectively optimizing for a different feature space on each client. In output personalization, it is the backpropagated gradients from the personalized layers that influence the shared layers. Since these gradients are conditioned on personalized outputs (e.g., client-specific classification heads), they differ across clients even for similar inputs, thereby causing divergence in the shared representation layer updates.

FedAPM can be considered a general FL framework, while FedAlt and FedSim are special cases of FedAPM. Specifically, when the dual variables $\boldsymbol{\pi}_i, i \in [m]$ and the penalty parameter ρ are set to $\mathbf{0}$, and the initial values of $\boldsymbol{u}_i$ are set to $\boldsymbol{u}^t$ in the corresponding subproblem, FedAPM reduces to FedAlt and FedSim. Therefore, FedAlt and FedSim can be interpreted as FL approaches that solve (3) through penalty methods. However, client drift arises in these methods:

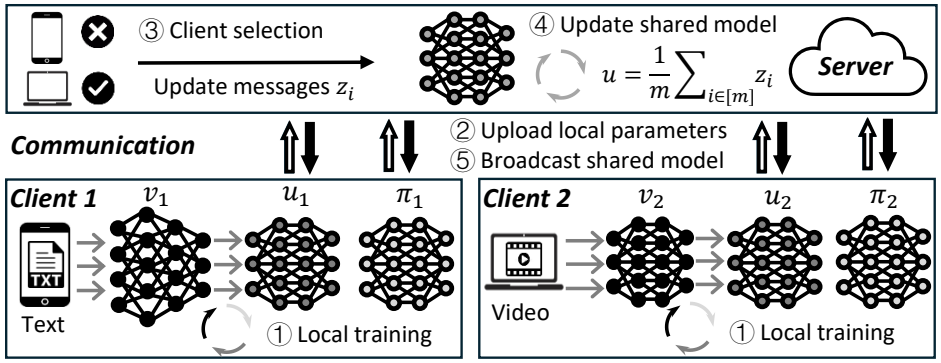


Figure 3: A running example of FedAPM. Different clients may possess heterogeneous or multimodal data. Each client updates its local parameters using local data and uploads z_i to the server, which then updates and broadcasts u to all clients.

each client optimizes its own objective rather than the global one due to differing local objectives, which could hinder convergence or even lead to divergence. While increasing ρ can mitigate this issue, it introduces another challenge: for ill-conditioned problems, an excessively large ρ is required to enforce constraints effectively. This, in turn, may result in numerical instability during optimization, as demonstrated in [51]. This issue is particularly problematic in the FL optimization, where the number of constraints m further increases the condition number of the objective function's Hessian matrix, exacerbating the risk of ill-conditioning.

In FedAPM, the second-order term provides a fixed correction for the update of $\boldsymbol{u}_i$, causing the gradient descent direction of $\boldsymbol{u}_i$ to shift towards $\boldsymbol{u}$ by a fixed value $\rho(\boldsymbol{u}_i - \boldsymbol{u})$, while the first-order term allows the gradient descent direction of $\boldsymbol{u}_i$ to shift towards $\boldsymbol{u}$ by a variable value $\boldsymbol{\pi}_i$, thereby providing compensation that brings $\boldsymbol{u}_i$ closer to $\boldsymbol{u}$ and effectively addressing the issue of client drift. Moreover, the update of the dual variables corrects the bias in constraint violations, allowing ADMM to avoid relying on large ρ to enforce the constraints, thus preventing ill-conditioning.

5 Convergence Analysis

We aim to establish the global convergence and convergence rate for FedAPM, starting with the assumptions used in our analysis.

5.1 Main Assumptions

We present the definitions of *graph*, *semicontinuous*, *real analytic*, and *semialgebraic* functions, which are utilized in our assumptions.

Definition 3 (Graph)**.** *Let $f : \mathbb{R}^p \to \mathbb{R} \cup \{+\infty\}$ be an extended real-valued function, its graph is defined by*

$$Graph(f) := \{(\boldsymbol{x}, y) \in \mathbb{R}^p \times \mathbb{R} : y = f(\boldsymbol{x})\},$$

and its domain is defined by $dom(f) := \{\boldsymbol{x} \in \mathbb{R}^p : f(\boldsymbol{x}) < +\infty\}$. If f is a proper function, i.e., $dom(f) \neq \emptyset$, then the set of its global minimizers is defined by $\operatorname{argmin} f := \{\boldsymbol{x} \in \mathbb{R}^p : f(\boldsymbol{x}) = \inf f\}$.

Definition 4 (Semicontinuous [8])**.** *A function $f : \mathcal{X} \to \mathbb{R}$ is lower semicontinuous if for any $x_0 \in \mathcal{X}$, $\lim_{x\to x_0} \inf f(x) \geq f(x_0)$.*

Definition 5 (Real analytic [33])**.** *A function f is real analytic on an open set $\mathcal{X}$ in the real line if for any $x_0 \in \mathcal{X}$, $f(x)$ can be represented as $f(x) = \sum_{i=1}^{+\infty} a_i(x - x_0)^i$, where $\{a_i\}_{i=1}^{+\infty}$ are real numbers and the series is convergent to $f(x)$ for x in a neighborhood of x_0.*

Definition 6 (Semialgebraic set and function [8])**.**

a) A set $\mathcal{X}$ is called semialgebraic if it can be represented by

$$\mathcal{X} = \cup_{i=1}^{r} \cap_{j=1}^{s} \{\boldsymbol{x} \in \mathbb{R}^p : P_{ij}(\boldsymbol{x}) = 0, Q_{ij}(\boldsymbol{x}) > 0\},$$

where $P_{ij}(\cdot)$ and $Q_{ij}(\cdot)$ are real polynomial functions for $1 \leq i \leq r$ and $1 \leq j \leq s$.

b) A function f is called semialgebraic if $Graph(f)$ is semialgebraic.

As highlighted in [8, 45, 58], semialgebraic sets are closed under several operations, including finite unions, finite intersections, Cartesian products, and complements. Typical examples include polynomial functions, indicator functions of a semialgebraic set, and Euclidean norm. In the following, we outline the assumptions employed in our convergence analysis.

Assumption 1. *Suppose the expected loss functions $f_i, i \in [m]$ are*

a) proper lower semicontinuous and nonnegative functions, and
b) either real analytic or semialgebraic.

Assumption 2 (Gradient Lipschitz continuity)**.** *For each $i \in [m]$, the expected loss function $f_i(\boldsymbol{v}_i, \boldsymbol{u})$ is continuously differentiable. There exist constants L_u, L_v, L_{uv}, L_{vu} such that for each $i \in [m]$:*

- *$\nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_i)$ is L_v-Lipschitz with respect to $\boldsymbol{v}_i$ and L_{vu}-Lipschitz with respect to $\boldsymbol{u}_i$, i.e.*

$$\|\nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_1, \boldsymbol{u}_i) - \nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_2, \boldsymbol{u}_i)\| \leq L_v \|\boldsymbol{v}_1 - \boldsymbol{v}_2\|,$$
$$\|\nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_1) - \nabla_{\boldsymbol{v}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_2)\| \leq L_{vu} \|\boldsymbol{u}_1 - \boldsymbol{u}_2\|,$$

- *$\nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_i)$ is L_u-Lipschitz with respect to $\boldsymbol{u}_i$ and L_{uv}-Lipschitz with respect to $\boldsymbol{v}_i$, i.e.*

$$\|\nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_1) - \nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_i, \boldsymbol{u}_2)\| \leq L_u \|\boldsymbol{u}_1 - \boldsymbol{u}_2\|,$$
$$\|\nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_1, \boldsymbol{u}_i) - \nabla_{\boldsymbol{u}_i} f_i(\boldsymbol{v}_1, \boldsymbol{u}_i)\| \leq L_{uv} \|\boldsymbol{v}_1 - \boldsymbol{v}_2\|,$$

Assumption 3. *The expected loss functions $f_i, i \in [m]$ are coercive*[1].

According to [8, 33, 45, 58], common loss functions such as squared, logistic, hinge, and cross-entropy losses can be verified to satisfy Assumption 1. Assumption 2 is a standard assumption in the convergence analysis of FL, as noted in [52]. Assumption 3 is widely

[1]An extended-real-valued function $f : \mathbb{R}^p \to \mathbb{R} \cup \{+\infty\}$ is called coercive if and only if $f(\boldsymbol{x}) \to +\infty$ as $\|\boldsymbol{x}\| \to +\infty$.

applied to establish the convergence properties of optimization algorithms, as shown in [73, 74, 80]. Our assumptions are weaker than those used by `FedAlt` and `FedSim` [52], which include *gradient Lipschitz continuity, bounded variance, and bounded diversity.*

5.2 Global Convergence

Our global convergence analysis builds upon the analytical framework presented in [6], which relies on four essential components: the establishment of *sufficient descent* and *relative error* conditions, along with the verification of the *continuity condition* and the *KŁ property* of the objective function. Let $\mathcal{P}^t := (\boldsymbol{V}^t, \boldsymbol{U}^t, \boldsymbol{\Pi}^t, \boldsymbol{u}^t)$, we define the Lyapunov function $\tilde{\mathcal{L}}(\mathcal{P}^t)$ by adding to the original augmented Lagrangian the accuracy level of each $\boldsymbol{u}_i^t$:

$$\tilde{\mathcal{L}}(\mathcal{P}^t) := \mathcal{L}(\mathcal{P}^t) + \sum_{i=1}^{m} \frac{29}{\rho(1-\mu_i)} \xi_i^t. \tag{17}$$

With the Lyapunov function established, we can demonstrate that the sequence generated by ADMM converges to the stationary points defined in (7) and (6) (see Theorem 2). Below, we present the two key lemmas, with additional details provided in the Appendix. Specifically, the verification of the KŁ property for FL training models satisfying Assumption 1 is outlined in Proposition 1 (Appendix A.2), while the verification of the continuity condition is naturally satisfied by Assumption 1. Using Lemmas 1 and 2, Proposition 1, and Assumption 1, we establish Theorem 2.

Lemma 1 (Sufficient Descent)**.** *Suppose that Assumption 2 holds. Let $\{\mathcal{P}^t\}$ denote the sequence generated by Algorithm 1; let each client set the hyperparameters such that* $\max\{3\alpha_i L_u, 3\alpha_i L_{uv}\} \le \rho \le \frac{15}{8}\sigma_i$*, then for all $t \ge 0$, it holds that*

$$\tilde{\mathcal{L}}(\mathcal{P}^t) - \tilde{\mathcal{L}}(\mathcal{P}^{t+1}) \ge a\Sigma_p^{t+1}, \text{ where } a := \min\{\frac{\rho}{60}, \frac{\sigma_i}{2} - \frac{4\rho}{15}\}$$

$$\text{and } \Sigma_p^{t+1} := \sum_{i=1}^{m} (\|\boldsymbol{v}_i^{t+1} - \boldsymbol{v}_i^t\|^2 + \|\boldsymbol{u}_i^{t+1} - \boldsymbol{u}_i^t\|^2 + \|\boldsymbol{u}^{t+1} - \boldsymbol{u}^t\|^2).$$

Lemma 2 (Relative Error)**.** *Suppose that Assumption 2 holds. Let $\{\mathcal{P}^t\}$ denote the sequence generated by Algorithm 1, let $\Xi^t := \sum_{i=1}^m \xi_i^t$, $\tilde{\Xi}^{t+1} := \Xi^{t+1} - \Xi^t$, and let each client set the hyperparameters such that $\sigma_i \ge \alpha_i L_v$ and* $\max\{3\alpha_i L_u, 3\alpha_i L_{uv}\} \le \rho \le \frac{15}{8}\sigma_i$*, then for all $t \ge 0$, the following result holds:*

$$dist(\mathbf{0}, \partial\mathcal{L}(\mathcal{P}^t))^2 \le b(\Sigma_p^{t+1} + \tilde{\Xi}^{t+1}), \tag{18}$$

where $b := \max\{\frac{22}{5}\sigma_i^2 + \frac{4}{3}\rho^2 + \frac{8\rho}{15}, \frac{16}{3}\rho^2 + 2 + \frac{8\rho}{15}, \frac{48+4\rho}{\rho(1-\mu_i)}\}$*,* $dist(0, C) := \inf_{\boldsymbol{c} \in C} \|\boldsymbol{c}\|$ *for a set C, and*

$$\partial\mathcal{L}(\mathcal{P}^t) := (\{\nabla_{\boldsymbol{v}_i}\mathcal{L}\}_{i=1}^m, \{\nabla_{\boldsymbol{u}_i}\mathcal{L}\}_{i=1}^m, \{\nabla_{\boldsymbol{\pi}_i}\mathcal{L}\}_{i=1}^m, \{\nabla_{\boldsymbol{u}}\mathcal{L}\})(\mathcal{P}^t).$$

Note that the relative error condition contains an error term $\tilde{\Xi}^{t+1}$ on the right-hand side of the inequality. The existing theoretical frameworks [6, 67, 74] cannot account for this error term. In the following, we present an analysis framework of global convergence that incorporates this error term.

Theorem 1. *Suppose that Assumption 2 holds, let each client set the hyperparameters such that* $\max\{3\alpha_i L_u, 3\alpha_i L_{uv}\} \le \rho \le \frac{15}{8}\sigma_i$ *and $\sigma_i \ge \alpha_i L_v$, then the following results hold.*

a) Sequence $\{\mathcal{P}^t\}$ is bounded.

b) The gradients of $f(\boldsymbol{V}^{t+1}, \boldsymbol{U}^{t+1})$ and $f(\boldsymbol{V}^{t+1}, \boldsymbol{u}^{t+1})$ with respect to each variable eventually vanish, i.e.,

$$\begin{aligned} \lim_{t\to\infty} \nabla_{\boldsymbol{V}} f(\boldsymbol{V}^{t+1}, \boldsymbol{u}^{t+1}) &= \lim_{t\to\infty} \nabla_{\boldsymbol{V}} f(\boldsymbol{V}^{t+1}, \boldsymbol{U}^{t+1}) \to 0, \\ \lim_{t\to\infty} \nabla_{\boldsymbol{u}} f(\boldsymbol{V}^{t+1}, \boldsymbol{u}^{t+1}) &= \lim_{t\to\infty} \nabla_{\boldsymbol{U}} f(\boldsymbol{V}^{t+1}, \boldsymbol{U}^{t+1}) \to 0. \end{aligned} \tag{19}$$

c) Sequences $\{\tilde{\mathcal{L}}(\mathcal{P}^t)\}$, $\{\mathcal{L}(\mathcal{P}^t)\}$, $\{f(\boldsymbol{V}^t, \boldsymbol{U}^t)\}$, and $\{f(\boldsymbol{V}^t, \boldsymbol{u}^t)\}$ converge to the same value, i.e.,

$$\lim_{t\to\infty} \tilde{\mathcal{L}}(\mathcal{P}^t) = \lim_{t\to\infty} \mathcal{L}(\mathcal{P}^t) = \lim_{t\to\infty} f(\boldsymbol{V}^t, \boldsymbol{U}^t) = \lim_{t\to\infty} f(\boldsymbol{V}^t, \boldsymbol{u}^t). \tag{20}$$

Theorem 1 establishes the convergence of the objective function. Next, we establish the convergence properties of the sequence $\{\mathcal{P}^t\}$.

Theorem 2. *Suppose that Assumptions 2 and 3 hold, let each client i set the hyperparameters such that* $\max\{3\alpha_i L_u, 3\alpha_i L_{uv}, 2L_u\} \le \rho \le \frac{15}{8}\sigma_i$ *and $\sigma_i \ge \alpha_i L_v$, then the following results hold:*

a) The accumulating point $\mathcal{P}^\infty$ of sequences $\{\mathcal{P}^t\}$ is a stationary point of (3)*, and $(\boldsymbol{V}^\infty, \boldsymbol{u}^\infty)$ is a stationary point of* (2)*.*

b) Under Assumption 1, the sequence $\{\mathcal{P}^t\}$ converges to $\mathcal{P}^\infty$.

Note that the proof of Theorem 2 does not depend on the convexity of the loss function f_i. As a result, the sequence will reach a stationary point for (3) and (2). Moreover, if we assume that f_i is convex, the sequence will converge to the optimal solutions.

5.3 Convergence Rate

We establish the convergence rates when $\tilde{\mathcal{L}}$ has KŁ properties with a desingularizing function $\phi(x) = \frac{\sqrt{c}}{1-\theta} x^{1-\theta}$ (see Definition 7 and 9 in Appendix A.2), where $c > 0$ and $\theta \in [0, 1)$. We elaborate in Proposition 1 that most functions in FL are KŁ functions.

Theorem 3. *Let $\{\mathcal{P}^t\}$ be the sequence generated by Algorithm 1, and $\mathcal{P}^\infty$ be its limit, then under Assumptions 1, 2, and 3, let each client set the hyperparameters* $\max\{3\alpha_i L_u, 3\alpha_i L_{uv}, 2L_u\} \le \rho \le \frac{15}{8}\sigma_i$ *and $\sigma_i \ge \alpha_i L_v$, the following results hold:*

a) If $\theta = 0$, then there exists a t_1 such that the sequence $\{\tilde{\mathcal{L}}(\mathcal{P}^t)\}$, $t \ge t_1$ converges in a finite number of iterations.

b) If $\theta \in (0, 1/2]$, then there exists a t_2 such that for any $t \ge t_2$,

$$\tilde{\mathcal{L}}(\mathcal{P}^{t+1}) - \tilde{\mathcal{L}}(\mathcal{P}^\infty) \le \left(\frac{bc}{a+bc}\right)^{t-t_2+1} \left(\tilde{\mathcal{L}}(\mathcal{P}^{t_2}) - f^*\right) + (a+bc)\Xi^{t_2-1}.$$

c) If $\theta \in (1/2, 1)$, then there exists a t_3 such that for any $t \ge t_3$,

$$\tilde{\mathcal{L}}(\mathcal{P}^{t+1}) - \tilde{\mathcal{L}}(\mathcal{P}^\infty) \le \left(\frac{bc}{(2\theta-1)\kappa a(t-t_3)}\right)^{\frac{1}{2\theta-1}},$$

where $\kappa > 0$ is a constant.

Theorem 3 demonstrates that when $\theta = 0$, the convergence rate becomes constant. For $\theta \in (0, 1/2]$, the rate is linear, while for $\theta \in (1/2, 1)$, it is sublinear. Note that although Theorem 3 relies on Assumptions 1, 2 and 3, these conditions are satisfied by most loss functions and are less restrictive compared to those in [52].

6 Numerical Experiments

In this section, we compare the performance of `FedAPM` with SOTA methods and examine the effects of various parameters on `FedAPM`. Our codes are open-sourced[2]. We present key design details and results, with complete content provided in Appendix B.

[2]https://github.com/whu-totemdb/FedAPM

Table 2: An overview of the datasets (Acc and Gyro are abbreviations of Accelerometer and Gyroscope, respectively).

Ref.	Datasets	Modalities	# samples	# clients
[34]	CIFAR10	Image	60,000	20
[3]	CrisisMMD	Image, Text	18,100	20
[59]	KU-HAR	Acc, Gyro	10,300	63
[13]	Crema-D	Audio, VIdeo	4,798	72

6.1 Setups

Datasets. We evaluate FedAPM as well as other comparable algorithms on four datasets from common heterogeneous and multimodal FL benchmarks [24, 38], which are CIFAR10 [34], CrisisMMD [3], KU-HAR [59], and Crema-D [13]. Among these, CIFAR10 is unimodal, while the others are multimodal. For each client, the training and testing data are pre-specified: 80% of data is randomly extracted to construct a training set, keeping the remaining 20% as the testing set. Table 2 presents an overview of the datasets.

Partitions. To incorporate data heterogeneity, we implement various data partitioning strategies as outlined in [24, 38]. For CIFAR10 and CrisisMMD, we allocate a portion of samples from each class to every client based on the Dirichlet distribution, which is commonly used for partitioning non-i.i.d. data [38]. For Crema-D and KU-HAR, we partition data by the speaker and participant IDs, respectively.

Models. We provide different classification models for various datasets. Specifically, we employ a CNN-only model architecture for the image modality, an RNN-only model for the video and text modalities, and a Conv-RNN model for other modalities.

Metrics and Baselines. We use the *accuracy, F1 score, AUC performance, training loss, gap between local and shared models, and communication rounds* as metrics to measure the performance of each competing method. To demonstrate the empirical performance of our proposed method, we compare FedAPM with two state-of-the-art personalization methods, FedAlt and FedSim [52], as well as two non-personalized methods, FedAvg [48] and FedProx [41].

6.2 Comparison of Multiple Methods

For each competing algorithm, different hyperparameters need to be tuned. We provide several candidates for each hyperparameter and perform a grid search on all possible combinations based on the accuracy performance on the validation dataset. Further details on parameter settings can be found in Appendix B.3.

6.2.1 Overall Comparison. Table 3 reports the top-1 accuracy, F1 score, and AUC for various methods. It can be observed that FedAPM outperforms all other methods across the datasets on most of the metrics. Specifically, compared to the best-performing method among the other four, FedAPM achieves an average improvement of 12.3%, 16.4%, and 18.0% in accuracy, F1 score, and AUC, respectively. This characteristic indicates that FedAPM is advantageous for FL applications involving heterogeneous or multimodal client data.

6.2.2 Convergence and Communication Efficiency. Due to space limitations, we show the results on two datasets here with the full results left in Figure 8 in Appendix B.5. Figure 4 shows the variation in training loss across different methods as the communication rounds increase. First, it can be observed that the methods based on partial model personalization (FedAPM, FedAlt, FedSim) tend to converge more effectively compared to non-personalized methods (FedAvg and FedProx). Second, FedAPM demonstrates lower training loss compared to other methods and achieves faster convergence, highlighting its superior convergence performance. Finally, for a given loss, FedAPM reaches this target with fewer communication rounds, emphasizing its communication efficiency over other methods.

Table 3: Performance comparison of various methods across multiple datasets. We report the mean and standard deviation for top-1 Accuracy, F1 Score, and AUC over 20 trials. Bold values highlight the best performance for each metric.

Datasets	Methods	Accuracy	F1 Score	AUC
CIFAR10	FedAPM	$\mathbf{.738_{\pm.005}}$	$\mathbf{.725_{\pm.005}}$	$\mathbf{.873_{\pm.010}}$
	FedAvg	$.435_{\pm.012}$	$.345_{\pm.010}$	$.602_{\pm.011}$
	FedAlt	$.672_{\pm.006}$	$.649_{\pm.005}$	$.839_{\pm.011}$
	FedSim	$.592_{\pm.005}$	$.565_{\pm.004}$	$.808_{\pm.010}$
	FedProx	$.406_{\pm.010}$	$.307_{\pm.005}$	$.571_{\pm.011}$
CrisisMMD	FedAPM	$.357_{\pm.028}$	$\mathbf{.294_{\pm.008}}$	$\mathbf{.519_{\pm.015}}$
	FedAvg	$.374_{\pm.023}$	$.288_{\pm.012}$	$.510_{\pm.027}$
	FedAlt	$.364_{\pm.025}$	$.289_{\pm.010}$	$.513_{\pm.019}$
	FedSim	$.364_{\pm.025}$	$.291_{\pm.009}$	$.514_{\pm.019}$
	FedProx	$\mathbf{.380_{\pm.017}}$	$.291_{\pm.009}$	$.507_{\pm.024}$
KU-HAR	FedAPM	$\mathbf{.437_{\pm.051}}$	$\mathbf{.396_{\pm.061}}$	$\mathbf{.595_{\pm.002}}$
	FedAvg	$.142_{\pm.004}$	$.050_{\pm.004}$	$.464_{\pm.023}$
	FedAlt	$.196_{\pm.017}$	$.105_{\pm.022}$	$.562_{\pm.006}$
	FedSim	$.196_{\pm.012}$	$.105_{\pm.017}$	$.566_{\pm.007}$
	FedProx	$.143_{\pm.005}$	$.049_{\pm.005}$	$.453_{\pm.023}$
Crema-D	FedAPM	$\mathbf{.651_{\pm.007}}$	$\mathbf{.590_{\pm.009}}$	$\mathbf{.695_{\pm.018}}$
	FedAvg	$.434_{\pm.025}$	$.295_{\pm.041}$	$.604_{\pm.044}$
	FedAlt	$.444_{\pm.031}$	$.304_{\pm.042}$	$.642_{\pm.020}$
	FedSim	$.444_{\pm.031}$	$.304_{\pm.042}$	$.643_{\pm.020}$
	FedProx	$.438_{\pm.032}$	$.297_{\pm.045}$	$.600_{\pm.046}$

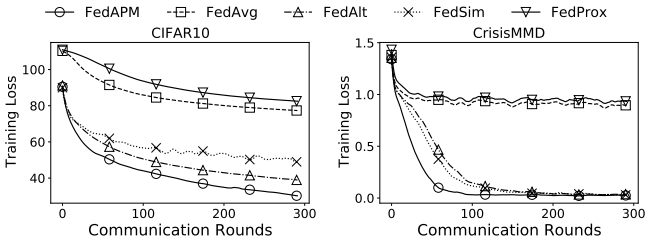


Figure 4: Comparison of training loss across various methods.

6.2.3 Model Deviation. Figure 5 illustrates the distance comparison between local and shared models for different methods [3]. First, partial model personalization can sometimes exacerbate client drift. In the KU-HAR dataset, both FedAlt and FedSim cause the local model

[3]For non-personalized methods, the shared model components we selected are the same as those in the personalized methods.

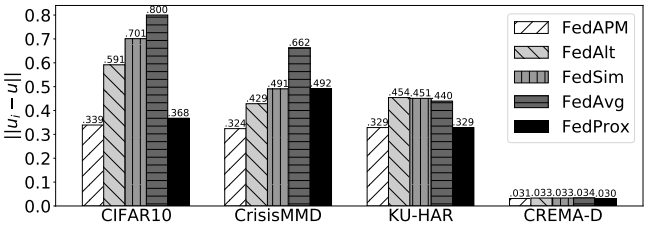


Figure 5: The distance between the local and the shared models across different FL methods. A larger distance indicates that the local model deviates further from the shared model.

to diverge further from the shared model compared to FedAvg and FedProx. Second, we observe that the local model trained with FedAPM is closer to the shared model than those trained with other methods, indicating that FedAPM can effectively address client drift.

6.3 Comparison of Multiple Hyperparameters

6.3.1 Effect of Penalty Parameter. We show the results on two datasets here with the full results shown in Figure 9 in Appendix B.5. Figure 6 illustrates how the training loss of FedAPM changes with communication rounds under different values of ρ, where ρ takes the values of $\{0.001, 0.01, 0.02, 0.05, 0.1\}$. The experimental results on CIFAR10 and CrisisMMD show that FedAPM effectively addresses ill-conditioning, with smaller ρ leading to faster convergence without the need for an excessively large ρ. This supports the analysis presented in Section 4.4, where we show that the dual variable effectively corrects the bias in constraint violations, allowing ADMM to avoid relying on large ρ to enforce the constraints.

6.3.2 Effect of Client Selection. We show the results on two datasets here with the full results presented in Figure 10 in Appendix B.5. Figure 7 illustrates the variation in the training loss of FedAPM with different numbers of selected clients, where the fraction of selected clients is $\{0.1, 0.2, 0.3, 0.5\}$. It can be observed that as the number of selected clients increases, the training loss decreases more rapidly. This is because a greater number of clients participating enables more frequent model updates, thereby enhancing model accuracy. Therefore, when communication capacity allows, increasing the number of selected clients can improve convergence speed.

6.4 Insights on Experimental Results

We have compared the performance of FedAPM with SOTA methods and validated the impact of hyperparameters on the performance of FedAPM. Our findings have led to the following insights:

- FedAPM demonstrates superior accuracy, F1 score, AUC, convergence, and communication efficiency compared to state-of-the-art methods in heterogeneous and multimodal settings.
- Partial model personalization can exacerbate client drift, causing the local model to diverge further from the shared model. The proposed FedAPM effectively mitigates client drift, bringing the local model closer to the shared model.
- Selecting appropriate hyperparameters for FedAPM can effectively enhance convergence. Specifically, choosing a relatively small value for ρ and increasing the number of clients participating in each training round can yield faster convergence.

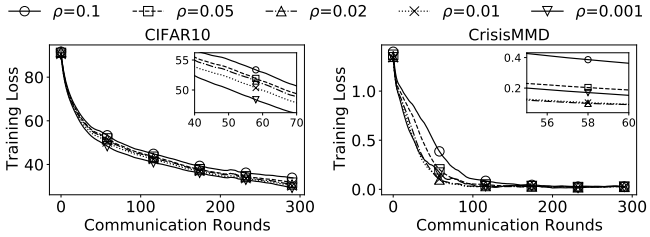


Figure 6: Convergence v.s. penalty parameter.

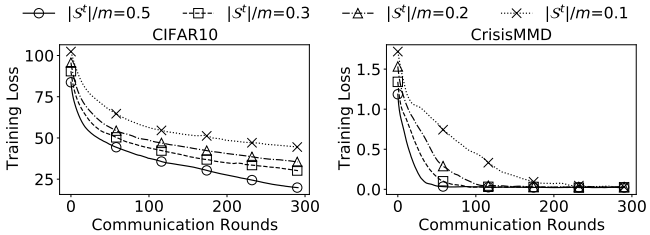


Figure 7: Convergence v.s. client selection.

7 Conclusions and Future Work

In this paper, we proposed an FL framework based on ADMM, called FedAPM, which performed well in heterogeneous and multimodal settings. FedAPM effectively addressed client drift and ill-conditioning issues in other FL frameworks. We established global convergence for FedAPM with three convergence rates under weaker conditions than existing theoretical frameworks. Our experimental results demonstrated the superior performance of FedAPM across multiple datasets and metrics, and we also validated the impact of different hyperparameters, providing a strategy for their selection.

In future work, we plan to improve FedAPM with robust privacy-preserving mechanisms, such as encryption techniques or differential privacy, to prevent adversaries from inferring users' sensitive information from the shared model parameters. Additionally, we will explore the theoretical foundations and practical applications of federated learning and ADMM methods within foundation models.

Acknowledgments

This work was supported by the National Key R&D Program of China (2023YFB4503600), National Natural Science Foundation of China (62202338), Key R&D Program of Hubei Province (2023BAB081), and Ant Group through CCF-Ant Research Fund. Jinshan Zeng was partially supported by the National Natural Science Foundation of China under Grant No.62376110, the Jiangxi Provincial Natural Science Foundation for Distinguished Young Scholars under Grant No. 20224ACB212004. Yao Yuan was partially supported by the Research Grants Council (RGC) of Hong Kong, SAR, China (GRF-16308321) and the NSFC/RGC Joint Research Scheme Grant N_HKUST635/20.

References

[1] 2018. California Consumer Privacy Act. https://en.wikipedia.org/wiki/California_Consumer_Privacy_Act.

[2] 2018. General Data Protection Regulation. https://en.wikipedia.org/wiki/General_Data_Protection_Regulation.

[3] Firoj Alam, Ferda Ofli, and Muhammad Imran. 2018. CrisisMMD: Multimodal Twitter Datasets from Natural Disasters. In *ICWSM*. 465–473.

[4] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. 2019. Federated learning with personalization layers. *arXiv preprint arXiv:1912.00818* (2019).

[5] Hédy Attouch and Jérôme Bolte. 2009. On the convergence of the proximal algorithm for nonsmooth functions involving analytic features. *Math. Program.* 116, 1-2 (2009), 5–16.

[6] Hédy Attouch, Jérôme Bolte, and Benar Fux Svaiter. 2013. Convergence of descent methods for semi-algebraic and tame problems: proximal algorithms, forward-backward splitting, and regularized Gauss-Seidel methods. *Math. Program.* 137, 1-2 (2013), 91–129.

[7] Yoshua Bengio, Aaron C. Courville, and Pascal Vincent. 2013. Representation Learning: A Review and New Perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* 35, 8 (2013), 1798–1828.

[8] Jacek Bochnak, Michel Coste, and Marie-Françoise Roy. 1998. *Real algebraic geometry.*

[9] Jérôme Bolte, Aris Daniilidis, and Adrian Lewis. 2007. The Łojasiewicz inequality for nonsmooth subanalytic functions with applications to subgradient dynamical systems. *SIAM J. Optim.* 17, 4 (2007), 1205–1223.

[10] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *SysML*.

[11] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, Jonathan Eckstein, et al. 2011. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Found. Trends Mach. Learn.* 3, 1 (2011), 1–122.

[12] Sebastian Caldas, Sai Meher Karthik Duddu, Peter Wu, Tian Li, Jakub Konečnỳ, H Brendan McMahan, Virginia Smith, and Ameet Talwalkar. 2018. Leaf: A benchmark for federated settings. *arXiv preprint arXiv:1812.01097* (2018).

[13] Houwei Cao, David G. Cooper, Michael K. Keutmann, Ruben C. Gur, Ani Nenkova, and Ragini Verma. 2014. CREMA-D: Crowd-Sourced Emotional Multimodal Actors Dataset. *IEEE Trans. Affect. Comput.* 5, 4 (2014), 377–390.

[14] Hong-You Chen and Wei-Lun Chao. 2022. On Bridging Generic and Personalized Federated Learning for Image Classification. In *ICLR*.

[15] Jiayi Chen and Aidong Zhang. 2022. FedMSplit: Correlation-Adaptive Federated Multi-Task Learning across Multimodal Split Networks. In *KDD*. 87–96.

[16] Jiayi Chen and Aidong Zhang. 2024. FedMBridge: Bridgeable Multimodal Federated Learning. In *ICML*.

[17] Li-Wei Chen and Alexander Rudnicky. 2023. Exploring wav2vec 2.0 fine tuning for improved speech emotion recognition. In *ICASSP*. 1–5.

[18] Kyunghyun Cho, Bart van Merrienboer, Çaglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *EMNLP*. 1724–1734.

[19] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. 2021. Exploiting Shared Representations for Personalized Federated Learning. In *ICML*, Vol. 139. 2089–2099.

[20] Ana Garcia del Molino and Michael Gygli. 2018. PHD-GIFs: Personalized Highlight Detection for Automatic GIF Creation. In *ACM MM*. 600–608.

[21] James Demmel. 1997. *Applied Numerical Linear Algebra.* SIAM.

[22] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[23] Canh T. Dinh, Nguyen Hoang Tran, and Tuan Dung Nguyen. 2020. Personalized Federated Learning with Moreau Envelopes. In *NeurIPS*. 21394–21405.

[24] Tiantian Feng, Digbalay Bose, Tuo Zhang, Rajat Hebbar, Anil Ramakrishna, Rahul Gupta, Mi Zhang, Salman Avestimehr, and Shrikanth Narayanan. 2023. FedMultimodal: A Benchmark for Multimodal Federated Learning. In *KDD*. 4035–4045.

[25] Yonghai Gong, Yichuan Li, and Nikolaos M. Freris. 2022. FedADMM: A Robust Federated Deep Learning Framework with Adaptivity to System Heterogeneity. In *ICDE*. 2575–2587.

[26] Ian Goodfellow. 2016. Deep learning.

[27] Filip Hanzely, Boxin Zhao, and Mladen Kolar. 2021. Personalized federated learning: A unified framework and universal optimization techniques. *arXiv preprint arXiv:2102.09743* (2021).

[28] Andrew G Howard. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).

[29] Peter Kairouz, Ziyu Liu, and Thomas Steinke. 2021. The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation. In *ICML*, Vol. 139. 5201–5212.

[30] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista A. Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaïd Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. 2021. Advances and Open Problems in Federated Learning. *Found. Trends Mach. Learn.* 14, 1-2 (2021), 1–210.

[31] Heejoo Kang, Minsoo Kim, Bumsuk Lee, and Hongseok Kim. 2024. FedAND: Federated Learning Exploiting Consensus ADMM by Nulling Drift. *IEEE Trans. Ind. Informatics* 20, 7 (2024), 9837–9849.

[32] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank J. Reddi, Sebastian U. Stich, and Ananda Theertha Suresh. 2020. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *ICML*, Vol. 119. 5132–5143.

[33] Steven G Krantz and Harold R Parks. 2002. *A primer of real analytic functions.*

[34] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[35] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. FederatedScope-LLM: A Comprehensive Package for Fine-tuning Large Language Models in Federated Learning. In *KDD*. ACM, 5260–5271.

[36] Krzysztof Kurdyka. 1998. On gradients of functions definable in o-minimal structures. In *Annales de l'institut Fourier*, Vol. 48. 769–783.

[37] Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. 2015. Deep learning. *Nat.* 521, 7553 (2015), 436–444.

[38] Qinbin Li, Yiqun Diao, Quan Chen, and Bingsheng He. 2022. Federated Learning on Non-IID Data Silos: An Experimental Study. In *ICDE*. 965–978.

[39] Tian Li, Shengyuan Hu, Ahmad Beirami, and Virginia Smith. 2021. Ditto: Fair and Robust Federated Learning Through Personalization. In *ICML*, Vol. 139. 6357–6368.

[40] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Process. Mag.* 37, 3 (2020), 50–60.

[41] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated Optimization in Heterogeneous Networks. In *MLSys*.

[42] Paul Pu Liang, Terrance Liu, Ziyin Liu, Ruslan Salakhutdinov, and Louis-Philippe Morency. 2020. Think Locally, Act Globally: Federated Learning with Local and Global Representations. In *NeurIPS*.

[43] Shiyun Lin, Yuze Han, Xiang Li, and Zhihua Zhang. 2022. Personalized Federated Learning towards Communication Efficiency, Robustness and Fairness. In *NeurIPS*. 30471–30485.

[44] Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. 2021. FATE: An Industrial Grade Platform for Collaborative Learning With Data Protection. *J. Mach. Learn. Res.* 22 (2021), 226:1–226:6.

[45] Stanis Lojasiewicz. 1965. Ensembles semi-analytiques. *Institut des Hautes Etudes Scientifiques* (1965).

[46] Mi Luo, Fei Chen, Dapeng Hu, Yifan Zhang, Jian Liang, and Jiashi Feng. 2021. No Fear of Heterogeneity: Classifier Calibration for Federated Learning with Non-IID Data. In *NeurIPS*. 5972–5984.

[47] Chenxin Ma, Virginia Smith, Martin Jaggi, Michael I. Jordan, Peter Richtárik, and Martin Takác. 2015. Adding vs. Averaging in Distributed Primal-Dual Optimization. In *ICML*, Vol. 37. 1973–1982.

[48] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*. 1273–1282.

[49] Tomás Mikolov and Geoffrey Zweig. 2012. Context dependent recurrent neural network language model. In *SLT*. 234–239.

[50] Boris S Mordukhovich. 2006. *Variational analysis and generalized differentiation I: Basic Theory.*

[51] Jorge Nocedal and Stephen J. Wright. 1999. *Numerical Optimization.* Springer.

[52] Krishna Pillutla, Kshitiz Malik, Abdelrahman Mohamed, Michael G. Rabbat, Maziar Sanjabi, and Lin Xiao. 2022. Federated Learning with Partial Model Personalization. In *ICML*. 17716–17758.

[53] Jian Ren, Xiaohui Shen, Zhe L. Lin, Radomír Mech, and David J. Foran. 2017. Personalized Image Aesthetics. In *ICCV*. 638–647.

[54] R Tyrrell Rockafellar and Roger J-B Wets. 1998. *Variational analysis.*

[55] Felix Sattler, Simon Wiedemann, Klaus-Robert Müller, and Wojciech Samek. 2020. Robust and Communication-Efficient Federated Learning From Non-i.i.d. Data. *IEEE Trans. Neural Networks Learn. Syst.* 31, 9 (2020), 3400–3413.

[56] Shai Shalev-Shwartz and Tong Zhang. 2013. Stochastic dual coordinate ascent methods for regularized loss. *J. Mach. Learn. Res.* 14, 1 (2013), 567–599.

[57] Shai Shalev-Shwartz and Tong Zhang. 2016. Accelerated proximal stochastic dual coordinate ascent for regularized loss minimization. *Math. Program.* 155, 1-2 (2016), 105–145.

[58] Masahiro Shiota. 1997. *Geometry of subanalytic and semialgebraic sets.*

[59] Niloy Sikder and Abdullah Al Nahid. 2021. KU-HAR: An open dataset for heterogeneous human activity recognition. *Pattern Recognit. Lett.* 146 (2021), 46–54.

[60] Karan Singhal, Hakim Sidahmed, Zachary Garrett, Shanshan Wu, John Rush, and Sushant Prakash. 2021. Federated Reconstruction: Partially Local Federated Learning. In *NeurIPS*. 11220–11232.

[61] Virginia Smith, Chao-Kai Chiang, Maziar Sanjabi, and Ameet Talwalkar. 2017. Federated Multi-Task Learning. In *NeurIPS*. 4424–4434.

[62] Virginia Smith, Simone Forte, Chenxin Ma, Martin Takác, Michael I. Jordan, and Martin Jaggi. 2017. CoCoA: A General Framework for Communication-Efficient Distributed Optimization. *J. Mach. Learn. Res.* 18 (2017), 230:1–230:49.

[63] Guangyu Sun, Matías Mendieta, Jun Luo, Shandong Wu, and Chen Chen. 2023. FedPerfix: Towards Partial Model Personalization of Vision Transformers in Federated Learning. In *ICCV*. 4965–4975.

[64] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2020. Mobilebert: a compact task-agnostic bert for resource-limited devices. *arXiv preprint arXiv:2004.02984* (2020).

[65] Chen Wang, Jialin Qiao, Xiangdong Huang, Shaoxu Song, Haonan Hou, Tian Jiang, Lei Rui, Jianmin Wang, and Jiaguang Sun. 2023. Apache IoTDB: A Time Series Database for IoT Applications. *Proc. ACM Manag. Data* 1, 2 (2023), 195:1–195:27.

[66] Han Wang, Siddartha Marella, and James Anderson. 2022. FedADMM: A federated primal-dual algorithm allowing partial participation. In *CDC*. IEEE, 287–294.

[67] Yu Wang, Wotao Yin, and Jinshan Zeng. 2019. Global convergence of ADMM in nonconvex nonsmooth optimization. *J. Sci. Comput.* 78, 1 (2019), 29–63.

[68] Liang Xiao, Xiaoyue Wan, Xiaozhen Lu, Yanyong Zhang, and Di Wu. 2018. IoT Security Techniques Based on Machine Learning: How Do IoT Devices Use AI to Enhance Security? *IEEE Signal Process. Mag.* 35, 5 (2018), 41–49.

[69] Yangyang Xu and Wotao Yin. 2013. A Block Coordinate Descent Method for Regularized Multiconvex Optimization with Applications to Nonnegative Tensor Factorization and Completion. *SIAM J. Imaging Sci.* 6, 3 (2013), 1758–1789.

[70] Piyush Yadav, Dhaval Salwala, Felipe Arruda Pontes, Praneet Dhingra, and Edward Curry. 2021. Query-Driven Video Event Processing for the Internet of Multimedia Things. *Proc. VLDB Endow.* 14, 12 (2021), 2847–2850.

[71] Mang Ye, Xiuwen Fang, Bo Du, Pong C. Yuen, and Dacheng Tao. 2024. Heterogeneous Federated Learning: State-of-the-art and Research Challenges. *ACM Comput. Surv.* 56, 3 (2024), 79:1–79:44.

[72] Rui Ye, Wenhao Wang, Jingyi Chai, Dihan Li, Zexi Li, Yinda Xu, Yaxin Du, Yanfeng Wang, and Siheng Chen. 2024. OpenFedLLM: Training Large Language Models on Decentralized Private Data via Federated Learning. In *KDD*. ACM, 6137–6147.

[73] Jinshan Zeng, Tim Tsz-Kit Lau, Shaobo Lin, and Yuan Yao. 2019. Global Convergence of Block Coordinate Descent in Deep Learning. In *ICML*, Vol. 97. 7313–7323.

[74] Jinshan Zeng, Shao-Bo Lin, Yuan Yao, and Ding-Xuan Zhou. 2021. On ADMM in Deep Learning: Convergence and Saturation-Avoidance. *J. Mach. Learn. Res.* 22 (2021), 199:1–199:67.

[75] Jianqing Zhang, Yang Hua, Hao Wang, Tao Song, Zhengui Xue, Ruhui Ma, and Haibing Guan. 2023. FedALA: Adaptive Local Aggregation for Personalized Federated Learning. In *AAAI*. 11237–11244.

[76] Jianqing Zhang, Yang Hua, Hao Wang, Tao Song, Zhengui Xue, Ruhui Ma, and Haibing Guan. 2023. FedCP: Separating Feature Information for Personalized Federated Learning via Conditional Policy. In *KDD*. 3249–3261.

[77] Xinwei Zhang, Mingyi Hong, Sairaj Dhople, Wotao Yin, and Yang Liu. 2021. FedPD: A federated learning framework with adaptivity to non-iid data. *IEEE Trans. Signal Process.* 69 (2021), 6055–6070.

[78] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. 2018. Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582* (2018).

[79] Shenglong Zhou and Geoffrey Ye Li. 2023. Federated Learning Via Inexact ADMM. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 8 (2023), 9699–9708.

[80] Shenglong Zhou and Geoffrey Ye Li. 2023. FedGiA: An Efficient Hybrid Algorithm for Federated Learning. *IEEE Trans. Signal Process.* 71 (2023), 1493–1508.

[81] Shengkun Zhu, Jinshan Zeng, Sheng Wang, Yuan Sun, Xiaodong Li, Yuan Yao, and Zhiyong Peng. 2024. On ADMM in Heterogeneous Federated Learning: Personalization, Robustness, and Fairness. *arXiv preprint arXiv:2407.16397* (2024).